Summarize

Part II. Single-Node Patterns

This book concerns itself with distributed systems, which are applications made up of many different components running on many different machines. However, the first section of this book is devoted to patterns that exist on a single node. The motivation for this is straightforward. Containers are the foundational building block for the patterns in this book, but in the end, groups of containers colocated on a single machine make up the atomic elements of distributed system patterns.

Though it is clear as to why you might want to break your distributed application into a collection of different containers running on different machines, it is perhaps somewhat less clear as to why you might also want to break up the components running on a single machine into different containers. To understand the motivation for these groups of containers, it is worth considering the goals behind containerization. In general, the goal of a container is to establish boundaries around specific resources (e.g., this application needs two cores and 8 GB of memory). Likewise, the boundary delineates team ownership (e.g., this team owns this image). Finally, the boundary is intended to provide separation of concerns (e.g., this image does this one thing).

All of these reasons provide motivation for splitting up an application on a single machine into a group of containers. Consider resource isolation first. Your application may be made up of two components: one is a user-facing application server and the other is a background configuration file loader. Clearly, end-user-facing request latency is the highest priority, so the user-facing application needs to have sufficient resources to ensure that it is highly responsive. On the other hand, the background configuration loader is mostly a best-effort service; if it is delayed slightly during times of high user-request volume, the system will be OK. Likewise, the background configuration loader should not impact the quality of service that end users receive. For all of these reasons, you want to separate the user-facing service and the background shard loader into different containers. This allows you to attach different resource requirements and priorities to the two different containers and, for example, ensure that the background loader opportunistically steals cycles from the user-facing service whenever it is lightly loaded and

the cycles are free. Likewise, separate resource requirements for the two containers ensure that the background loader will be terminated before the user-facing service if there is a resource contention issue caused by a memory leak or other overcommitment of memory resources.

In addition to this resource isolation, there are other reasons to split your single-node application into multiple containers. Consider the task of scaling a team. There is good reason to believe that the ideal team size is six to eight people. In order to structure teams in this manner and yet still build significant systems, we need to have small, focused pieces for each team to own. Additionally, often some of the components, if factored properly, are reusable modules that can be used by many teams.

Consider, for example, the task of keeping a local filesystem synchronized with a Git source code repository. If you build this Git sync tool as a separate container, you can reuse it with PHP, HTML, JavaScript, Python, and numerous other web-serving environments. If you instead factor each environment as a single container where, for example, the Python runtime and the Git synchronization are inextricably bound, then this sort of modular reuse (and the corresponding small team that owns that reusable module) is impossible.

Finally, even if your application is small and all of your containers are owned by a single team, the separation of concerns ensures that your application is well understood and can easily be tested, updated, and deployed. Small, focused applications are easier to understand and have fewer couplings to other systems. This means, for example, that you can deploy the Git synchronization container without having to also redeploy your application server. This leads to rollouts with fewer dependencies and smaller scope. That, in turn, leads to more reliable rollouts (and rollbacks), which leads to greater agility and flexibility when deploying your application.

I hope that all of these examples have motivated you to think about breaking up your applications, even those on a single node, into multiple containers. The following chapters describe some patterns that can help guide you as you build modular groups of containers. In contrast to later patterns we'll discuss, all of the patterns in this section assume tight dependencies between all containers. In particular, they assume that all of the containers in the pattern can be reliably coscheduled onto a single machine.

They also assume that all of the containers in the pattern can optionally share volumes or parts of their filesystems as well as other key container resources like network namespaces and shared memory. This tight grouping is called a *pod* in Kubernetes,[1] but the

concept is generally applicable to different container orchestrators, though some support it more natively than others.

---

[1] Kubernetes is an open source system for automating deployment, scaling, and management of containerized applications. Check out my book, *Kubernetes: Up and Running* (O'Reilly).

Previous chapter
< 2. Important Distributed System Concepts

Next chapter
3. The Sidecar Pattern >